💻 Clase 6 – Comparativa entre Lenguajes (Versión Fiel al Material)

Docentes:

- Ing. Sebastián Balestrini
- Esp. Ing. Gustavo Di Stefano

Objetivos

- Reconocer diferencias y similitudes entre distintos lenguajes
- Comparar estilos y estructuras básicas
- Comprender cómo un mismo problema se resuelve de formas distintas

---

Herramientas

- **OneCompiler** es similar a OnlineGDB, pero incluye FreeBasic: https://onecompiler.com/
- **OnlineGDB**: https://www.onlinegdb.com/

Comparativa entre Lenguajes (texto original)

- Los lenguajes clásicos son la base de la programación moderna.
- Cada uno surgió con un objetivo específico:
  - **C, Pascal, BASIC:** enseñanza y desarrollo estructurado.
  - **COBOL:** gestión de datos en empresas.
  - **Assembler:** control directo del hardware.
- Compararlos permite:
  - Reconocer similitudes en lógica (variables, condicionales, bucles).
  - Identificar diferencias de sintaxis y propósito.
- Idea clave: **“Distintos caminos para resolver los mismos problemas”**.

Panorama de los lenguajes clásicos

🕰️ Línea de tiempo de lenguajes de programación

Año	Lenguaje	Contexto o propósito principal
1843	Primer lenguaje	Ada Lovelace escribe el primer algoritmo para una máquina
1957	Fortran	Primer lenguaje de alto nivel, orientado a cálculos científicos
1958	LISP	Diseñado para inteligencia artificial
1959	COBOL	Enfocado en aplicaciones de negocio

Año	Lenguaje	Contexto o propósito principal
1964	BASIC	Pensado para principiantes
1969	C	Creado para sistemas Unix
1970	Pascal	Promueve la programación estructurada
1972	C++	Extiende C con programación orientada a objetos
1980	Ada	Lenguaje estructurado, usado en sistemas críticos
1983	Extensión de C	Evolución del lenguaje C
1987	Perl	Potente para procesamiento de texto en Unix
1991	Python	Enfocado en legibilidad y productividad
1993	Ruby	Orientado a objetos, sintaxis elegante
1995	Java	Diseñado para portabilidad en múltiples plataformas
1995	JavaScript	Lenguaje de scripting para la web
1995	PHP	Dinamismo en páginas web
2000	ActionScript	Usado en animaciones Flash
2001	Visual Basic	Lenguaje visual para desarrollo rápido
2011	Dart	Creado por Google para aplicaciones web modernas

COBOL - Lenguaje de negocios (según material)

- Diseñado en 1959 → "Common Business Oriented Language".
- Enfocado en:
  - Procesamiento de grandes volúmenes de datos.
  - Aplicaciones de bancos, seguros, administración.
- Sintaxis cercana al inglés natural → pensado para programadores y analistas de gestión.
- Ventaja: claridad y robustez en operaciones comerciales.

Ejemplo (según material): sumar dos números en COBOL

```
IDENTIFICATION DIVISION.
PROGRAM-ID. SUMA.
DATA DIVISION.
WORKING-STORAGE SECTION.
01 A PIC 9(3).
01 B PIC 9(3).
01 RESULTADO PIC 9(3).

PROCEDURE DIVISION.
    DISPLAY "Ingrese primer número: " ACCEPT A.
    DISPLAY "Ingrese segundo número: " ACCEPT B.
```

```
    ADD A TO B GIVING RESULTADO.
    DISPLAY "La suma es: " RESULTADO.
STOP RUN.
```

Assembler – Lenguaje de bajo nivel (según material)

- Lenguaje más cercano al hardware de la CPU.
- Cada instrucción corresponde a una operación de máquina.
- Usado en: programación de microcontroladores y sistemas embebidos, drivers y control de periféricos, optimización de código crítico (velocidad, memoria).
- Ventaja: máximo control y eficiencia.
- Desventaja: poca portabilidad y mayor complejidad.

Ejemplo (según material): sumar dos números en x86 assembly

```
section .data
    a db 5
    b db 3
    res db 0

section .text
    global _start

_start:
    mov al, [a]
    add al, [b]
    mov [res], al

    ; Exit (Linux syscall)
    mov eax, 60
    xor edi, edi
    syscall
```

Pascal – Lenguaje estructurado para enseñanza (según material)

- Creado en 1970 por Niklaus Wirth.
- Diseñado para enseñar programación estructurada.
- Sintaxis clara, estricta y orientada a buenas prácticas.
- Usado en educación y primeras aplicaciones de escritorio.
- Favorece: declaración explícita de tipos y variables; estructuras de control clásicas: if, case, for, while.

Ejemplo (según material): suma de dos números en Pascal

```
program Suma;
var
  a, b, resultado: integer;
```

```
begin
  writeln('Ingrese dos números:');
  readln(a, b);
  resultado := a + b;
  writeln('La suma es: ', resultado);
end.
```

Funciones y Procedimientos en Pascal (según material)

- Procedimientos (procedure): ejecutan acciones, no devuelven valor, se llaman por su nombre.
- Funciones (function): ejecutan acciones y devuelven un valor, tienen un tipo de retorno, se pueden usar dentro de expresiones.

Ejemplo (según material): function y procedure en Pascal

```
function Sumar(x, y: integer): integer;
begin
  Sumar := x + y;
end;

procedure MostrarResultado;
var
  r: integer;
begin
  r := Sumar(10, 5);
  writeln('Resultado: ', r);
end;
```

Pasaje de parámetros en Pascal (según material)

- Tipos de pasaje en Pascal:
  - Por valor (default): se pasa una copia; los cambios no afectan la variable original.
  - Por referencia (var): se pasa la dirección; los cambios dentro del procedimiento afectan la variable original.

Ejemplo (según material): pasaje por referencia en Pascal

```
procedure Duplicar(var n: integer);
begin
  n := n * 2;
end;

var
  valor: integer = 5;
begin
  Duplicar(valor);
```

```
  writeln('Valor duplicado: ', valor);
end.
```

Basic – Lenguaje para aprender rápido (según material)

- Popular en los 80s–90s.
- Diseñado para ser fácil de usar y accesible en PC.
- Ideal para principiantes y pequeños programas.
- Sintaxis muy directa.
- Muy usado en entornos educativos y software sencillo.
- Característica criticada: GOTO (salto incondicional).
- QBasic permite programación estructurada; incorpora SUB y FUNCTION; evita el uso de GOSUB y números de línea.

Ejemplo en FreeBasic (según material)

```
PRINT "Ingrese dos números:"
INPUT a, b
resultado = a + b
PRINT "La suma es"; resultado
END
```

Funciones y Procedimientos en Basic (según material)

- Objetivo: modularizar el código, reutilizar lógica y mejorar la legibilidad.
  - Una función se declara con `Function` y finaliza con `End Function`.
  - Puede recibir parámetros y devolver un valor.
  - El tipo de retorno se indica después de `As`.
  - Un procedimiento (Sub) ejecuta acciones y no devuelve valor.

Ejemplo en FreeBasic (según material): Function y Sub

```
Function Sumar(a As Integer, b As Integer) As Integer
    Sumar = a + b
End Function

Sub Mostrar()
    Dim resultado As Integer
    resultado = Sumar(5, 10)
    Print "El resultado es "; resultado
End Sub

Mostrar()
Sleep
```

Pasaje de parámetros en Basic (según material)

- Entender cómo viajan los datos a la función:
  - Por valor (`ByVal`): se pasa una copia del dato.
  - Por referencia (`ByRef`): se pasa la dirección de memoria, útil para modificar el valor original.

Ejemplo en FreeBasic (según material): ByRef

```
Sub Duplicar(ByRef n As Integer)
    n = n * 2
End Sub

Dim x As Integer = 5
Duplicar(x)
Print "Nuevo valor: "; x
Sleep
```

Comparativa de lenguajes estructurados (tabla según material)

Aspecto	Pascal	BASIC / QuickBasic	C
Entrada/Salida	`readln`, `writeln`	`INPUT`, `PRINT`	`scanf`, `printf`
Condicionales	`if ... then ... else`, `case`	`IF ... THEN ... ELSE`	`if ... else`, `switch`
Bucles	`for ... to ... do`, `while ... do`	`FOR`, `WHILE`, `DO ... LOOP`	`for`, `while`, `do ... while`
Funciones	`function`, `procedure`	`Sub` y `Function`	funciones con prototipos
Estructura	Estricta y clara	Muy flexible y simple	Compacta, eficiente, cercana al hardware